

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования

Московский государственный технический университет имени
Н.Э.Баумана

Отчет по лабораторной работе №6

по теме:

Задача условного экстремума

Выполнил: студент
группы ИУ5-65Б
Рябова Софья Андреевна

Содержание

1	Цель работы	2
2	Постановка задачи	2
3	Функция Лагранжа	2
4	Анализ допустимости	3
4.1	Оба ограничения пассивны	3
4.2	Активно только ограничение g_1	3
4.3	Активно только ограничение g_2	3
4.4	Активны оба ограничения	4
5	Проверка второго порядка	4
6	Численное исследование	4
6.1	Результаты вычислений	5
6.2	Графический результат	5
7	Текст исходного кода программы	5
8	Заключение	10

1 Цель работы

Цель работы — исследовать задачу условного экстремума функции нескольких переменных с ограничениями, применить метод множителей Лагранжа и условия Куна—Таккера, а также выполнить численную проверку решения с использованием программы на Python.

2 Постановка задачи

Рассматривается задача условного экстремума для функции

$$f(x_1, x_2) = e^{x_1} - x_2^2 + 3 \ln(x_2 - 1) \rightarrow \text{extr}$$

при ограничениях

$$\begin{aligned} g_1(x_1, x_2) &= x_2 - x_1 \leq 0, \\ g_2(x_1, x_2) &= x_1^2 + x_2^2 - 10 \leq 0, \\ x_2 &> 1. \end{aligned}$$

Требуется:

1. составить функцию Лагранжа;
2. записать необходимые условия Куна—Таккера;
3. найти стационарные точки;
4. проверить допустимость найденных решений;
5. определить тип экстремума по второму дифференциалу;
6. выполнить численное исследование с помощью программы и представить результаты в графическом виде.

3 Функция Лагранжа

Классическая функция Лагранжа имеет вид:

$$L(x_1, x_2, \lambda_1, \lambda_2) = e^{x_1} - x_2^2 + 3 \ln(x_2 - 1) + \lambda_1(x_2 - x_1) + \lambda_2(x_1^2 + x_2^2 - 10).$$

Условия стационарности:

$$\begin{aligned} \frac{\partial L}{\partial x_1} &= e^{x_1} - \lambda_1 + 2\lambda_2 x_1 = 0, \\ \frac{\partial L}{\partial x_2} &= -2x_2 + \frac{3}{x_2 - 1} + \lambda_1 + 2\lambda_2 x_2 = 0. \end{aligned}$$

Условия Куна—Таккера:

$$\begin{aligned} \lambda_1 &\geq 0, & \lambda_2 &\geq 0, \\ g_1(x_1, x_2) &\leq 0, & g_2(x_1, x_2) &\leq 0, \\ \lambda_1 g_1(x_1, x_2) &= 0, & \lambda_2 g_2(x_1, x_2) &= 0. \end{aligned}$$

4 Анализ допустимости

Рассмотрим возможные случаи активности ограничений.

4.1 Оба ограничения пассивны

Если

$$\lambda_1 = 0, \quad \lambda_2 = 0,$$

то из первого условия стационарности получаем

$$e^{x_1} = 0,$$

что невозможно. Следовательно, внутренних стационарных точек нет.

4.2 Активно только ограничение g_1

Пусть

$$x_2 = x_1 = t, \quad \lambda_2 = 0.$$

Тогда из условий стационарности получаем уравнение

$$e^t - 2t + \frac{3}{t-1} = 0.$$

Для допустимости необходимо:

$$t > 1, \quad 2t^2 \leq 10,$$

то есть

$$1 < t \leq \sqrt{5}.$$

Численный поиск показывает существование допустимой стационарной точки

$$t \approx 1.64944.$$

Тогда

$$x^* = (1.64944, 1.64944),$$

$$\lambda_1 \approx 5.20389.$$

Значение функции в этой точке:

$$f(x^*) \approx 1.18856.$$

4.3 Активно только ограничение g_2

Пусть

$$\lambda_1 = 0, \quad x_1^2 + x_2^2 = 10.$$

Тогда система условий стационарности приводит к необходимости численного решения. Однако при анализе знаков множителей и условий допустимости выясняется, что допустимых стационарных точек в этом случае нет.

4.4 Активны оба ограничения

Если

$$x_2 = x_1, \quad x_1^2 + x_2^2 = 10,$$

то

$$x_1 = x_2 = \sqrt{5}.$$

Подстановка в условия Куна—Таккера показывает, что соответствующий множитель λ_2 оказывается отрицательным, что противоречит условию

$$\lambda_2 \geq 0.$$

Следовательно, этот случай также не дает допустимого решения.

5 Проверка второго порядка

Матрица Гессе функции Лагранжа по переменным x_1, x_2 :

$$\nabla_{xx}^2 L = \begin{pmatrix} e^{x_1} + 2\lambda_2 & 0 \\ 0 & -2 - \frac{3}{(x_2 - 1)^2} + 2\lambda_2 \end{pmatrix}.$$

Второй дифференциал:

$$d^2L = (e^{x_1} + 2\lambda_2) dx_1^2 + \left(-2 - \frac{3}{(x_2 - 1)^2} + 2\lambda_2\right) dx_2^2.$$

Если активно только ограничение g_1 , то

$$dg_1 = dx_2 - dx_1 = 0 \quad \Rightarrow \quad dx_2 = dx_1 = h.$$

Тогда

$$d^2L = \left(e^{x_1} - 2 - \frac{3}{(x_2 - 1)^2}\right) h^2.$$

Если

$$e^{x_1} - 2 - \frac{3}{(x_2 - 1)^2} < 0,$$

то точка является точкой условного локального максимума.

Если

$$e^{x_1} - 2 - \frac{3}{(x_2 - 1)^2} > 0,$$

то точка является точкой условного локального минимума.

6 Численное исследование

Для проверки аналитических выводов была написана программа на Python, которая:

- проверяет случаи активности ограничений;
- численно ищет стационарные точки;
- проверяет допустимость решений;
- вычисляет значение функции в найденной точке;
- строит график допустимой области и отмечает стационарные точки.

6.1 Результаты вычислений

По результатам численного исследования получено:

$$x^* = (1.64944, 1.64944),$$

$$f(x^*) \approx 1.18856.$$

Эта точка удовлетворяет ограничениям задачи и является допустимой стационарной точкой.

6.2 Графический результат

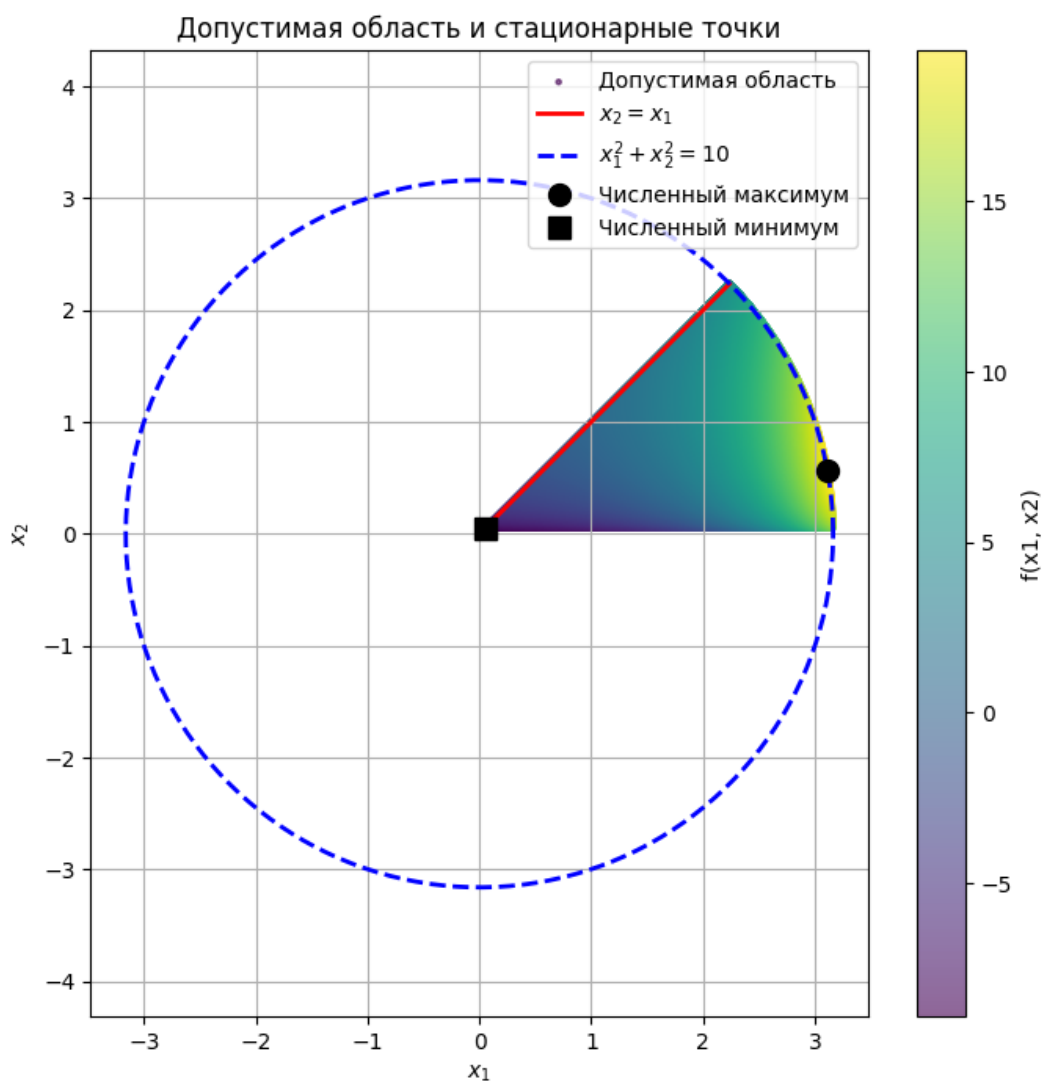


Рис. 1: Допустимая область и стационарные точки

7 Текст исходного кода программы

Листинг 1: Исходный код программы

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

```

3 from scipy.optimize import fsolve, minimize_scalar
4
5 # 14
6 # f(x1, x2) = exp(x1) - x2^2 + 3 ln(x2) - 1
7 # g1(x1, x2) = x2 - x1 <= 0
8 # g2(x1, x2) = x1^2 + x2^2 - 10 <= 0
9 # x2 > 0
10
11 def f(x1, x2):
12     if x2 <= 0:
13         return np.nan
14     return np.exp(x1) - x2**2 + 3*np.log(x2) - 1
15
16 def g1(x1, x2):
17     return x2 - x1
18
19 def g2(x1, x2):
20     return x1**2 + x2**2 - 10
21
22 def is_feasible(x1, x2, eps=1e-8):
23     return (x2 > 0) and (g1(x1, x2) <= eps) and (g2(x1, x2) <= eps)
24
25 # 1:
26 # lambda1 = lambda2 = 0
27 # exp(x1)=0 impossible
28 print(" 1:
29     -> ")
30
31 # 2: g1
32 # x2 = x1 = t
33 # exp(t) - 2t + 3/t = 0
34 def eq_case2(t):
35     return np.exp(t) - 2*t + 3/t
36
37 # (0, sqrt(5)]
38 grid = np.linspace(0.05, np.sqrt(5), 2000)
39 vals = eq_case2(grid)
40
41 roots_case2 = []
42 for i in range(len(grid)-1):
43     if np.isnan(vals[i]) or np.isnan(vals[i+1]):
44         continue
45     if vals[i] == 0:
46         roots_case2.append(grid[i])
47     elif vals[i]*vals[i+1] < 0:
48         root = fsolve(eq_case2, (grid[i] + grid[i+1]) / 2)[0]
49         if 0 < root <= np.sqrt(5):
50             if not any(abs(root-r) < 1e-5 for r in roots_case2):
51                 roots_case2.append(root)
52
53 print("\n 2: g1")
54 if len(roots_case2) == 0:
55     print(" ")

```

```

55 else:
56     for t in roots_case2:
57         x1, x2 = t, t
58         lam1 = np.exp(t)
59         val = f(x1, x2)
60         second = np.exp(t) - 2 - 3/(t**2)
61         print(f"t = {t:.6f}")
62         print(f"x* = ({x1:.6f}, {x2:.6f})")
63         print(f"lambda1 = {lam1:.6f}, lambda2 = 0")
64         print(f"f(x*) = {val:.6f}")
65         print(f"d2L along feasible direction = ({second:.6f}) h^2")
66         if second < 0:
67             print("=>
68                                     ")
69         elif second > 0:
70             print("=>
71                                     ")
72         else:
73             print("=>
74                                     ")
75
76 #                                     3:                                     g2
77 # lambda1 = 0
78 # x1^2 + x2^2 = 10
79 def system_case3(vars):
80     x1, x2, lam2 = vars
81     return [
82         np.exp(x1) + 2*lam2*x1,
83         -2*x2 + 3/x2 + 2*lam2*x2,
84         x1**2 + x2**2 - 10
85     ]
86
87 print("\n                                     3:                                     g2")
88 guesses = [
89     (-2.0, 1.0, 1.0),
90     (-1.5, 2.0, 0.5),
91     (-3.0, 1.0, 2.0),
92     (1.0, 3.0, 1.0),
93 ]
94 found_case3 = []
95
96 for guess in guesses:
97     try:
98         sol = fsolve(system_case3, guess)
99         x1, x2, lam2 = sol
100         if np.isfinite(x1) and np.isfinite(x2) and np.isfinite(lam2):
101             if abs(system_case3(sol)[0]) < 1e-6 and abs(system_case3(
102                 sol)[1]) < 1e-6 and abs(system_case3(sol)[2]) < 1e-6:
103                 duplicate = False
104                 for s in found_case3:
105                     if np.linalg.norm(sol - s) < 1e-5:
106                         duplicate = True
107                         break

```



```

104         if not duplicate:
105             found_case3.append(sol)
106     except:
107         pass
108
109 if len(found_case3) == 0:
110     print(" ")
111 else:
112     feasible_found = False
113     for sol in found_case3:
114         x1, x2, lam2 = sol
115         print(f" : x1={x1:.6f}, x2={x2:.6f}, lambda2={
116             lam2:.6f}")
117         print(f" : g1={g1(x1,
118             x2):.6f}, g2={g2(x1,x2):.6f}")
119         if is_feasible(x1, x2) and abs(g1(x1, x2)) < 1e-6 and lam2 >=
120             -1e-6:
121             feasible_found = True
122     if not feasible_found:
123         print(" ")
124
125 #
126 # x1 = x2, x1^2 + x2^2 = 10 => x1 = x2 = sqrt(5)
127 print("\n 4: g1 g2")
128 x1 = np.sqrt(5)
129 x2 = np.sqrt(5)
130
131 A = np.array([
132     [-1, 2*x1],
133     [ 1, 2*x2]
134 ], dtype=float)
135
136 b = np.array([
137     -np.exp(x1),
138     2*x2 - 3/x2
139 ], dtype=float)
140
141 lam1, lam2 = np.linalg.solve(A, b)
142
143 print(f"x* = ({x1:.6f}, {x2:.6f})")
144 print(f"lambda1 = {lam1:.6f}, lambda2 = {lam2:.6f}")
145
146 if lam1 >= -1e-8 and lam2 >= -1e-8:
147     print(" ")
148 else:
149     print(" ")
150
151 #
152 # x2 in (0, sqrt(5)], x1 in [x2, sqrt(10 - x2^2)]

```

```

150 # /
151 print("\n
           ")
152
153 x2_vals = np.linspace(0.05, np.sqrt(5), 500)
154 X1, X2 = [], []
155 F = []
156
157 for x2 in x2_vals:
158     x1_min = x2
159     rad = 10 - x2**2
160     if rad < 0:
161         continue
162     x1_max = np.sqrt(rad)
163     if x1_min > x1_max:
164         continue
165     x1_vals = np.linspace(x1_min, x1_max, 500)
166     for x1 in x1_vals:
167         if is_feasible(x1, x2):
168             X1.append(x1)
169             X2.append(x2)
170             F.append(f(x1, x2))
171
172 X1 = np.array(X1)
173 X2 = np.array(X2)
174 F = np.array(F)
175
176 imax = np.nanargmax(F)
177 imin = np.nanargmin(F)
178
179 print(f"           : x=({X1[imax]:.6f}, {X2[
    imax]:.6f}), f={F[imax]:.6f}")
180 print(f"           : x=({X1[imin]:.6f}, {X2[
    imin]:.6f}), f={F[imin]:.6f}")
181
182 #
183 plt.figure(figsize=(8, 8))
184
185 plt.scatter(X1, X2, c=F, s=5, cmap='viridis', alpha=0.6, label='
    ')
186
187 # x2 = x1
188 line = np.linspace(0, np.sqrt(5), 300)
189 plt.plot(line, line, 'r-', lw=2, label=r'$x_2=x_1$')
190
191 # x1^2 + x2^2 = 10
192 theta = np.linspace(0, 2*np.pi, 500)
193 cx = np.sqrt(10)*np.cos(theta)
194 cy = np.sqrt(10)*np.sin(theta)
195 plt.plot(cx, cy, 'b--', lw=2, label=r'$x_1^2+x_2^2=10$')
196
197 #

```

```

198 plt.plot(X1[imax], X2[imax], 'ko', ms=10, label='
      ')
199 plt.plot(X1[imin], X2[imin], 'ks', ms=10, label='
      ')
200
201 # 2
202 for t in roots_case2:
203     plt.plot(t, t, 'mo', ms=8, label='
      (g1      )')
204
205 plt.xlabel(r'$x_1$')
206 plt.ylabel(r'$x_2$')
207 plt.title('
      ')
208 plt.grid(True)
209 plt.axis('equal')
210 plt.legend()
211 plt.colorbar(label='f(x1, x2)')
212 plt.show()

```

8 Заключение

В работе была исследована задача условного экстремума с двумя неравенствами. Были составлены условия Куна—Таккера, выполнена проверка допустимости, проведен анализ второго порядка и реализована численная проверка на Python. Получена допустимая стационарная точка и определено значение целевой функции в этой точке.